

SISTEMAS OPERATIVOS II

2026

TEMAS – UNIDAD 2

2

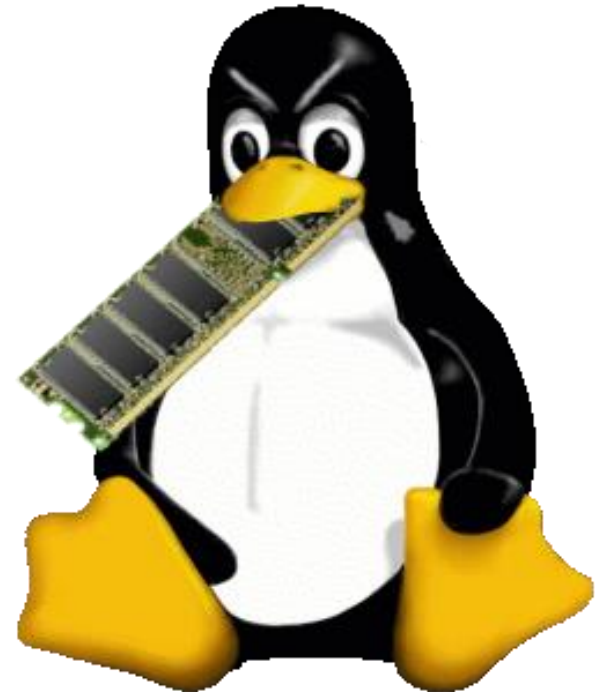
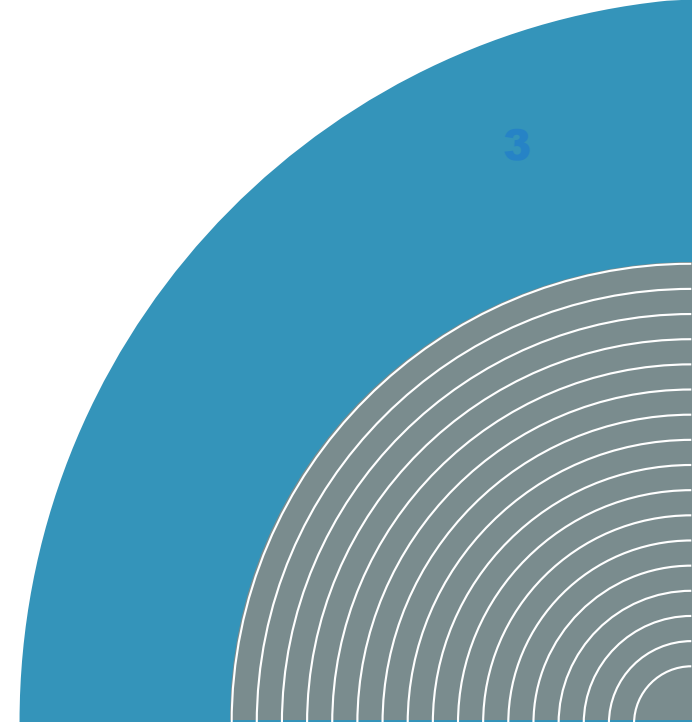
Administración de Memoria:

- Conceptos Fundamentales.
- Llamadas al sistema de administración de memoria.
- Implementación de la administración de memoria.
- Paginación

ADMINISTRACIÓN DE MEMORIA EN LINUX

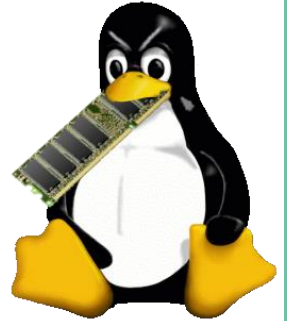
Objetivos

- Comprender la organización de la memoria en Linux.
- Analizar el espacio de direcciones de un proceso.
- Estudiar los mecanismos de asignación.
- Comprender la paginación y el reemplazo de páginas.
- Analizar el funcionamiento de Buddy System y Slab Allocator.



ADMINISTRACIÓN DE MEMORIA EN LINUX

4



¿Por qué es importante la administración de memoria?

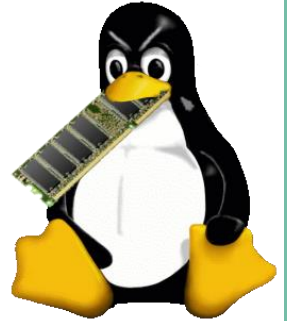
Un sistema operativo debe:

- Ejecutar múltiples procesos simultáneamente.
- Aprovechar eficientemente la RAM.
- Proteger procesos entre sí.
- Compartir recursos cuando sea necesario.
- Mantener alto rendimiento.

Linux proporciona a cada proceso la ilusión de poseer una memoria propia y continua.

ADMINISTRACIÓN DE MEMORIA EN LINUX

5



Espacio de direcciones de un proceso

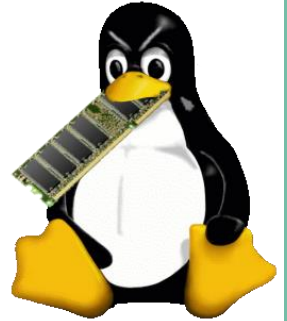
Todo proceso posee tres segmentos principales:

- Texto
- Datos
- Pila

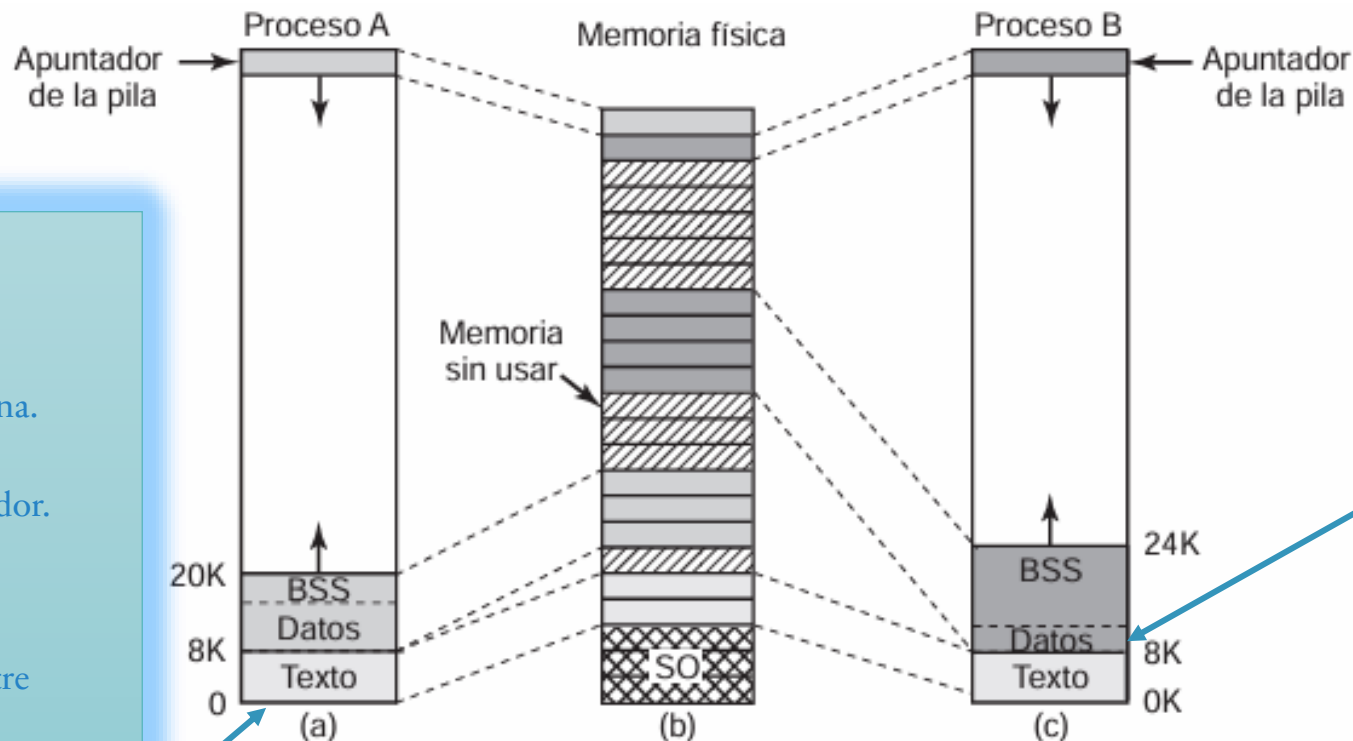
Además aparece una zona especial:

- Heap (Montículo)

ADMINISTRACIÓN DE MEMORIA EN LINUX



Estructura general de Memoria



Segmento de Texto

Contiene:

- Código ejecutable.
- Instrucciones de máquina.
- Código generado por compilador y ensamblador.

Características:

- ✓ Solo lectura.
- ✓ Puede compartirse entre procesos.
- ✓ No cambia durante la ejecución.

Segmento de Datos

Almacena:

- Variables globales.
- Constantes.
- Arreglos.
- Cadenas.

Se divide en:

Datos inicializados

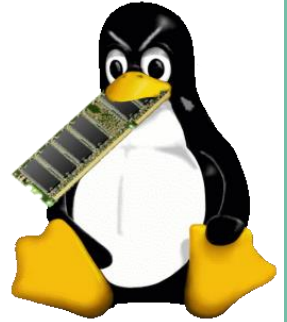
```
int edad = 20;
```

BSS

```
int contador;
```

ADMINISTRACIÓN DE MEMORIA EN LINUX

7



¿Qué es el BSS?

Linux no guarda miles de ceros en el ejecutable.

En lugar de ello:

- Reserva espacio.
- Inicializa los datos en cero al iniciar el programa.

Ventajas:

- Ejecutables más pequeños.
- Menor uso de disco.

Página Cero (Zero Page)

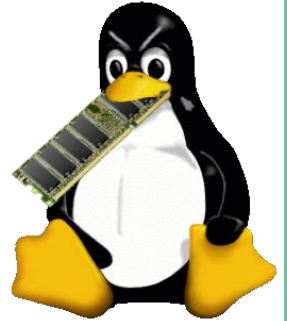
Para optimizar memoria:

- Linux mantiene una página llena de ceros.
- Protegida contra escritura.
- Compartida inicialmente.

Cuando un proceso intenta escribir:

→ **Se activa Copy-On-Write.**

ADMINISTRACIÓN DE MEMORIA EN LINUX



¿Qué es el BSS?

Linux no guarda miles de ceros en el ejecutable.

En lugar de ello:

- Reserva espacio.
- Inicializa los datos en cero al iniciar el programa.

Ventajas:

- Ejecutables más pequeños.
- Menor uso de disco.

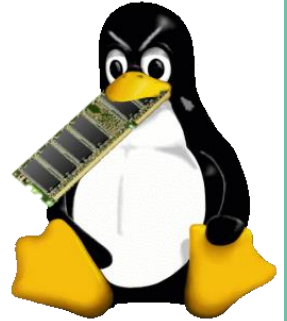
Página Cero (Zero Page)

Para optimizar memoria:

- Linux mantiene una página llena de ceros.
- Protegida contra escritura.
- Compartida inicialmente.

Cuando un proceso intenta escribir:

→ **Se activa Copy-On-Write.**



ADMINISTRACIÓN DE MEMORIA EN LINUX

Heap (Montículo)

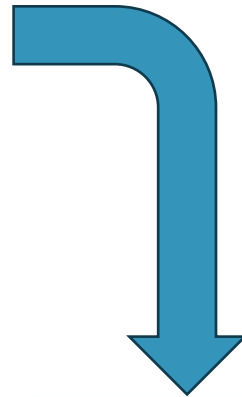
- Memoria dinámica.

Funciones asociadas:

```
1 malloc()
2 calloc()
3 realloc()
4 free()
```

Se administra mediante:

```
1 brk()
```

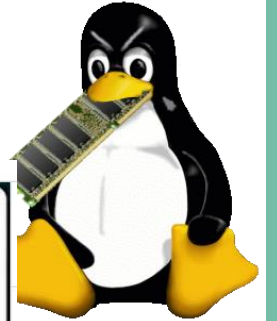


Características:

- Crece y disminuye dinámicamente.
- Utilizado durante la ejecución.

ADMINISTRACIÓN DE MEMORIA EN LINUX

10



Segmento de Pila (Stack)

Contiene:

- Variables locales.
- Parámetros.
- Direcciones de retorno.

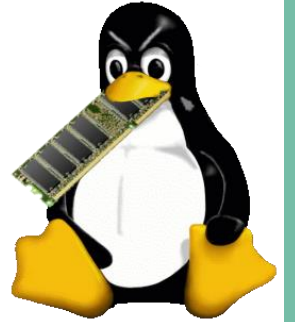
Características:

- ✓ Crece hacia abajo.
- ✓ Administración automática.
- ✓ Muy rápida.



ADMINISTRACIÓN DE MEMORIA EN LINUX

11



Compartición de Código

Dos procesos pueden compartir:

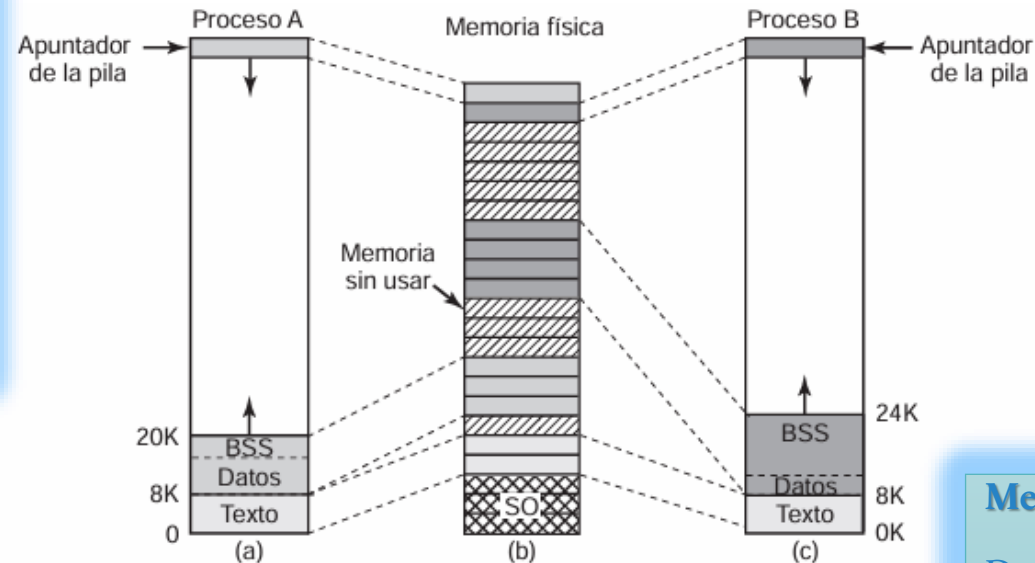
✓ Segmento de texto.

No comparten normalmente:

✗ Datos.

✗ Pila.

Esto reduce el consumo de memoria.



Memoria Compartida

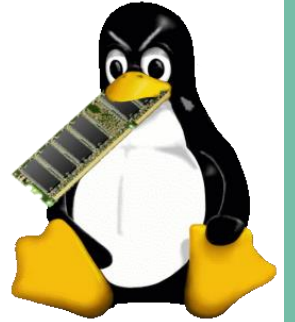
Dos procesos ejecutando el mismo programa:

El código se almacena una sola vez en RAM.

Los datos permanecen privados

ADMINISTRACIÓN DE MEMORIA EN LINUX

12



Archivos Mapeados en Memoria

Linux permite asociar un archivo a una región de memoria.

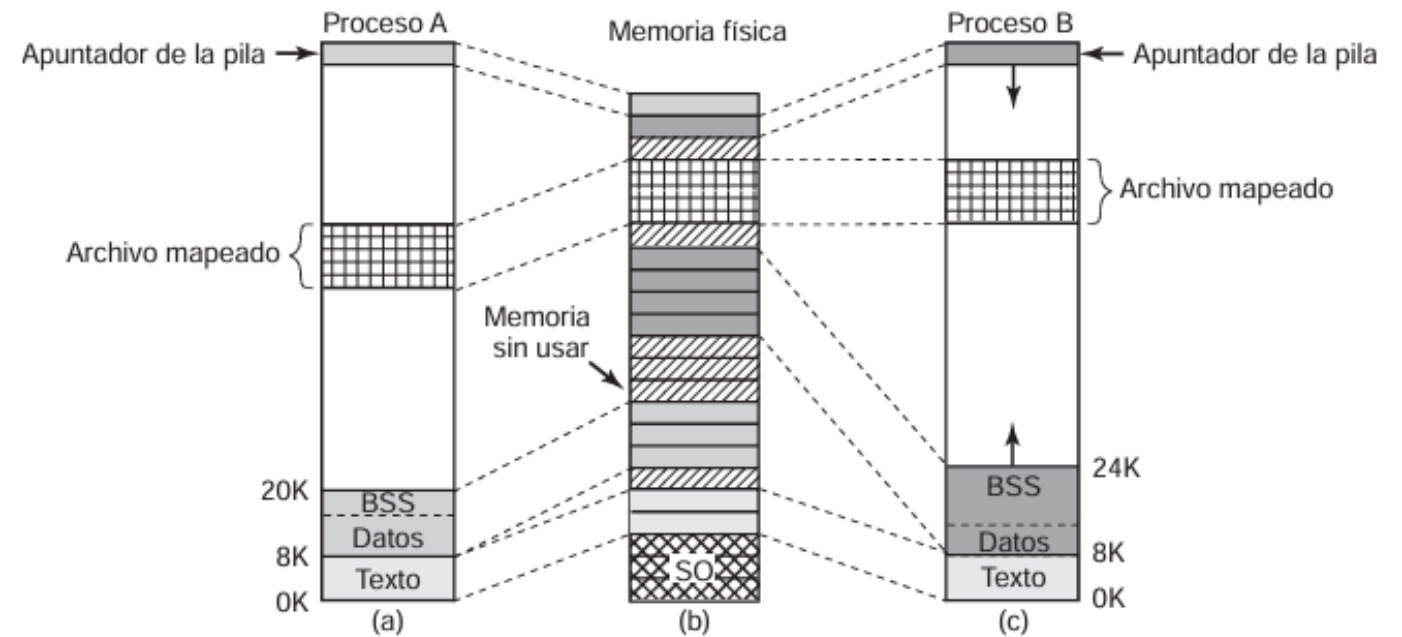
Llamada principal: `mmap()`

Ventajas de mmap()

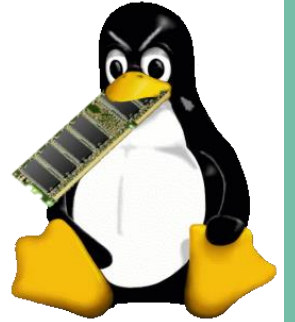
- Acceso rápido.
- Acceso aleatorio eficiente.
- Compartición entre procesos.
- Soporte para bibliotecas compartidas.

Archivo Compartido por Dos Procesos

Una modificación realizada por un proceso puede ser visible para otro proceso que comparte el mismo archivo mapeado



ADMINISTRACIÓN DE MEMORIA EN LINUX



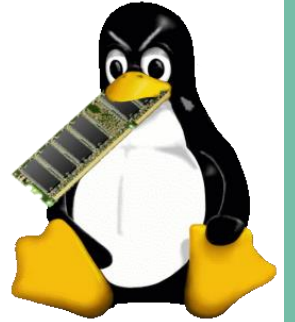
Llamadas al sistema para Administración de Memoria

- **POSIX** no especifica llamadas al sistema para Administración de Memoria
- Los programas que requieren la administración de memoria dinámica pueden utilizar el procedimiento de biblioteca “malloc” (definido por el estándar ANSI C)



Llamada al sistema	Descripción
s = brk(direc)	Cambia el tamaño del segmento de datos
a = mmap(direc, lon, prot, banderas, fd, despl)	Asigna un archivo
s = unmap(direc, lon)	Desasigna un archivo

IMPLEMENTACIÓN DE MEMORIA EN LINUX



Espacio Virtual en Linux (32 bits)

Cada proceso dispone de:

- 3 GB de espacio de direcciones virtuales para si mismo.
- 1GB se reserva para sus tablas de páginas y otros datos del Kernel

MEMORIA FÍSICA

ZONE_DMA

Páginas que se pueden utilizar para operaciones DMA

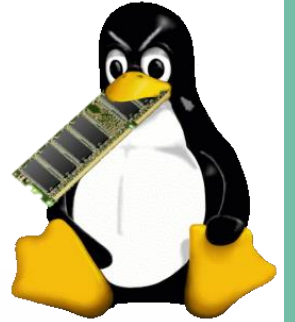
ZONE_NORMAL

Páginas normales que se asignan de la manera usual

ZONE_HIGHMEM

Páginas con direcciones de memoria superior, que no se asignan de manera permanente

IMPLEMENTACIÓN DE MEMORIA EN LINUX



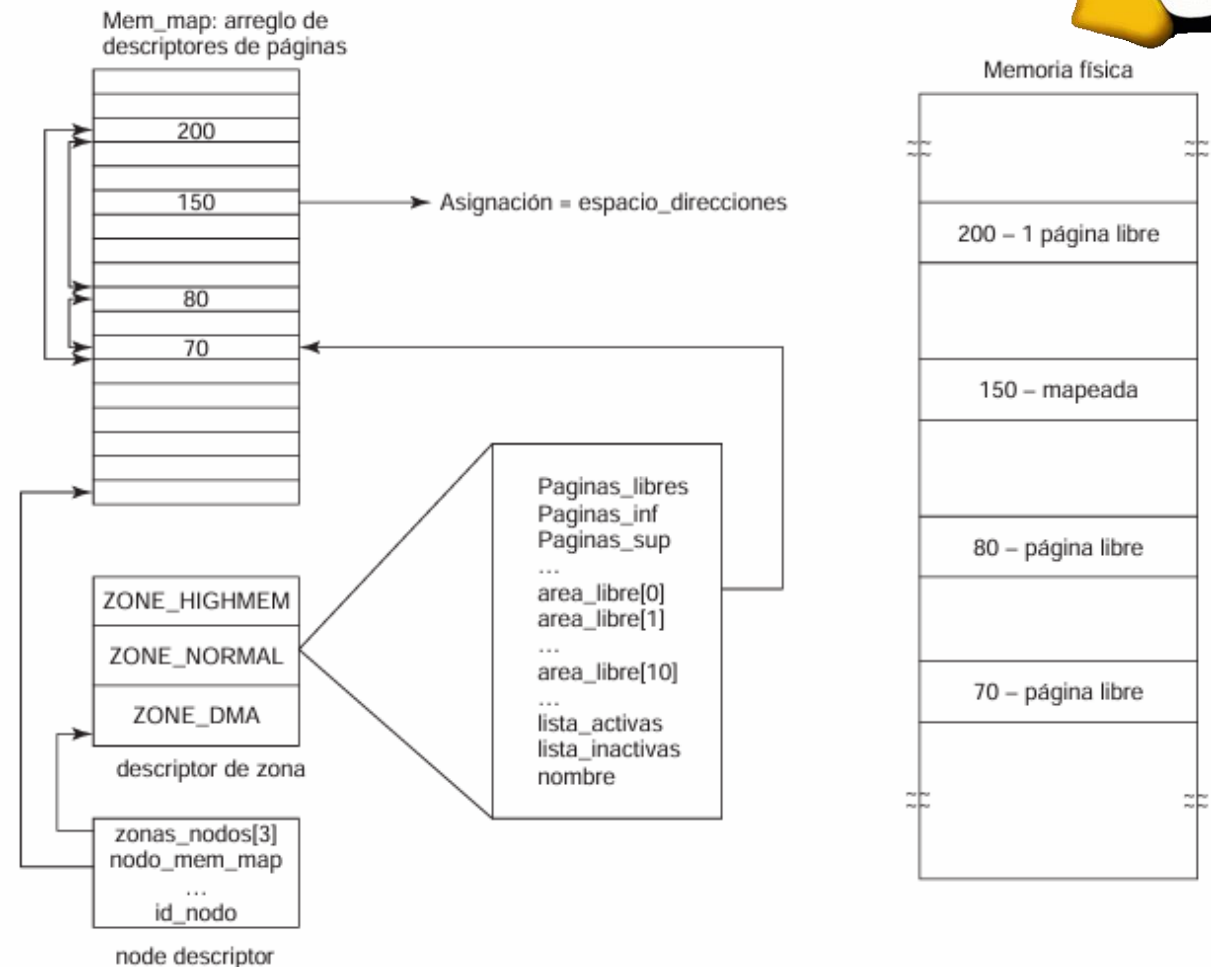
Organización de la memoria FISICA

Consiste en tres partes:

- Kernel
 - Mapa de Memoria
 - Resto Marcos de Páginas (Texto, Datos o Pila)
- } Residentes

Estructuras principales:

- mem_map: Descriptores de páginas
- Descriptor de página
- Descriptor de zona
- Descriptor de nodo



IMPLEMENTACIÓN DE MEMORIA EN LINUX

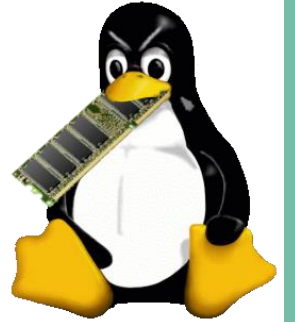


Tabla de Páginas de Cuatro Niveles

Linux utiliza:

- Directorio Global
- Directorio Superior
- Directorio Intermedio
- Tabla de Páginas

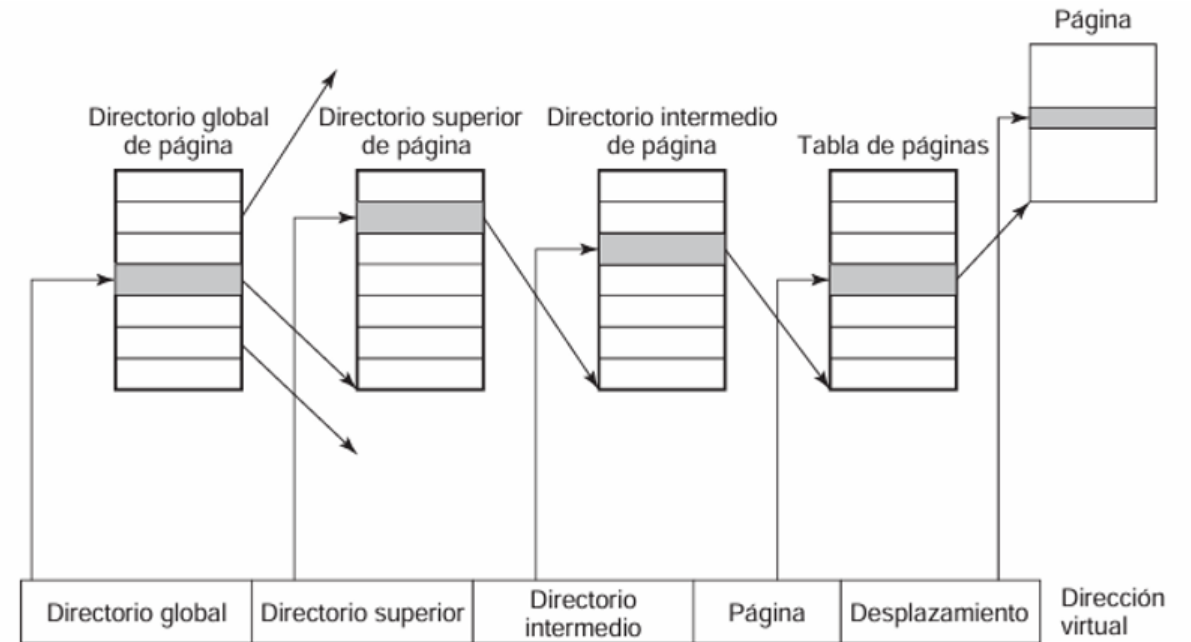
Objetivo:

Reducir memoria usada por las tablas.

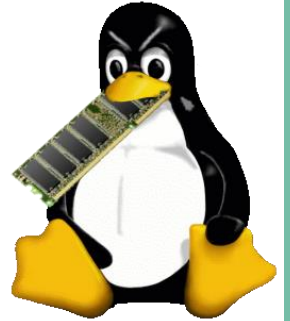
Soportar espacios grandes.

Esquema de Traducción

Dirección Virtual → Tabla de páginas → Marco físico.



IMPLEMENTACIÓN DE MEMORIA EN LINUX



Asignación de Memoria

Linux utiliza tres mecanismos:

- Buddy Allocator.
- Slab Allocator.
- vmalloc.

Buddy Allocator.

Idea: Dividir memoria en bloques potencia de 2.:

Funcionamiento del Buddy

Cuando dos bloques hermanos quedan libres:

→ Se fusionan nuevamente.

Ventajas y Desventajas del Buddy

- Ventajas
- Muy rápido.
- Fácil implementación.

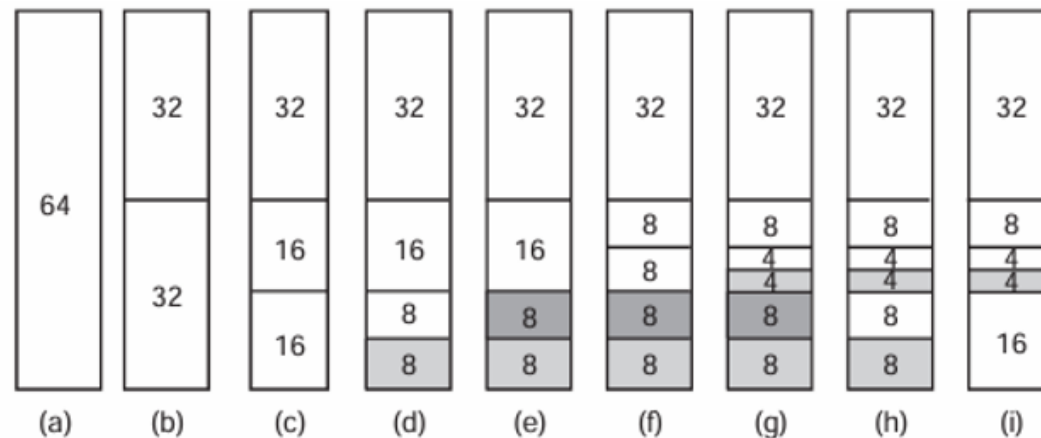
Desventajas

- Fragmentación interna.

Ejemplo:

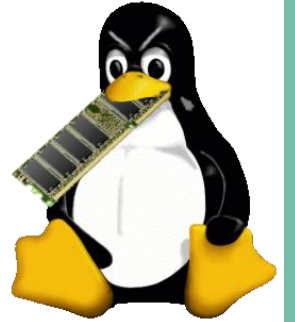
Solicitud = 65 páginas

Asignación = 128 páginas.



Se entrega el bloque necesario.

IMPLEMENTACIÓN DE MEMORIA EN LINUX



Slab Allocator

Objetivo:

Reducir la fragmentación interna del Buddy.

Concepto:

- Mantiene cachés de objetos frecuentes.
- Reutiliza estructuras ya preparadas.

Ejemplos:

- task_struct
- buffers
- descriptores del kernel

vmalloc()

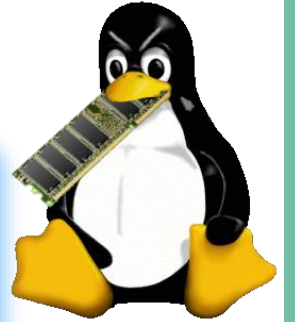
Asigna memoria:

- ✓ Contigua virtualmente
- ✗ No necesariamente contigua físicamente

Usos:

- Grandes regiones de memoria.
- Módulos del kernel.

IMPLEMENTACIÓN DE MEMORIA EN LINUX



Paginación bajo demanda *Demand Paging*:

Las páginas ingresan a memoria:

- ✓ Sólo cuando se necesitan.

Reduce:

- Tiempo de carga.
- Consumo de RAM.

Demonio de Paginación kswapd

Responsable de:

- Vigilar memoria libre.
- Liberar páginas.
- Mantener reservas disponibles

Swap:

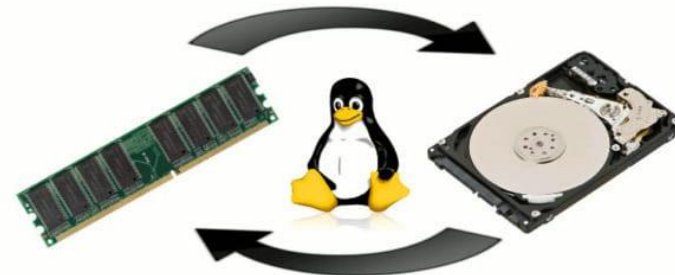
Cuando la memoria escasea:

RAM → Disco

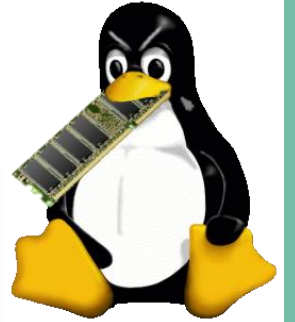
Opciones:

- Partición swap.
- Archivo swap.

La partición es más eficiente.



IMPLEMENTACIÓN DE MEMORIA EN LINUX



Algoritmo de Reemplazo de Página

Page Frame Reclaiming Algorithm.

Clasifica páginas como:

- No reclamables
- Intercambiables
- Sincronizables
- Descartables

Objetivo:

- Recuperar marcos de página.

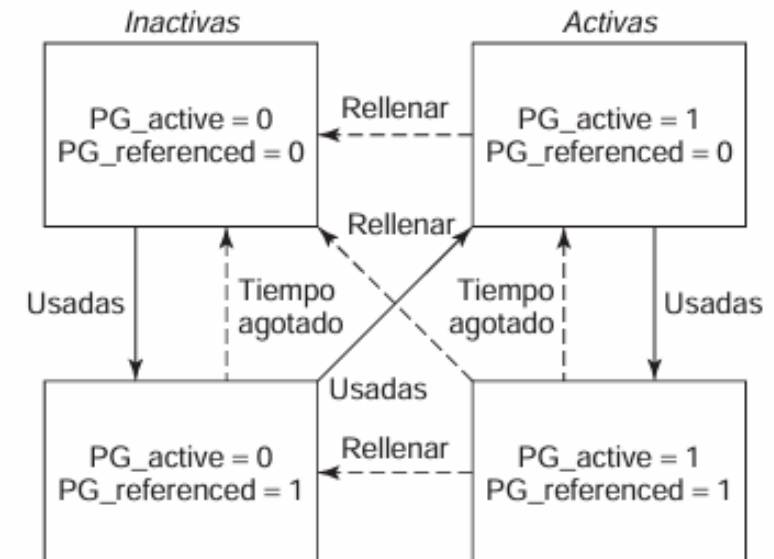
Estados:

- Activa / No activa
- Referenciada / No referenciada

Linux utiliza una variante del algoritmo Clock.

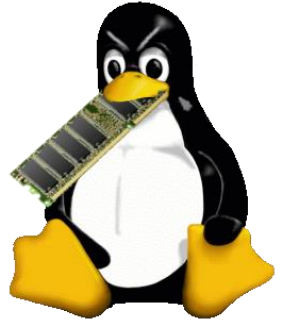
Las primeras candidatas a reemplazo son:

- 1 PG_active = 0
- 2 PG_referenced = 0



RESUMEN GENERAL

21



- ✓ Espacio de direcciones: Texto, Datos, BSS, Heap y Stack.
- ✓ Archivos mapeados.
- ✓ Buddy Allocator.
- ✓ Slab Allocator.
- ✓ vmalloc.
- ✓ Demand Paging.
- ✓ Swap.
- ✓ PFRA